

Language Models

Lê Thanh Hương

School of Information and Communication Technology

Email: huonglt@soict.hust.edu.vn

Language Models

- Compute the probability of a sentence or sequence of words in a language
- A good language model will consider: grammatically correct sentences are more fluently than words with a random order.
- E.g.: $P(\text{"hôm nay trời đẹp"}) > P(\text{"trời đẹp nay hôm"})$

N-gram Language Models

- Goal: compute the probability of a sentence or sequence of words:

$$P(W) = P(w_1, w_2, w_3, w_4, w_5, \dots, w_m)$$

- Recall the definition of conditional probabilities

$$p(B|A) = P(A,B)/P(A) \quad \text{Rewriting:} \quad P(A,B) = P(A)P(B|A)$$

- More variables:

$$P(A,B,C,D) = P(A)P(B|A)P(C|A,B)P(D|A,B,C)$$

- The Chain Rule in General

$$P(w_1, w_2, w_3, w_4, w_5, \dots, w_m) = P(w_1) * P(w_2 | w_1) * P(w_3 | w_1 w_2) * \dots * P(w_m | w_1 w_2 w_3 \dots w_{m-1})$$

$$P(\text{“hôm nay trời đẹp”}) =$$

$$P(\text{hôm}) * P(\text{nay|hôm}) * P(\text{trời|hôm nay}) * P(\text{đẹp |hôm nay trời})$$

How to estimate these probabilities?

$$P(\text{đẹp} | \text{hôm nay trời}) = \frac{P(\text{hôm nay trời đẹp})}{P(\text{hôm nay trời})}$$

- It is impossible to store all such probabilities, especially with m being the length of the natural language text
- Use Markov chain assuming that a word depends only on $n-1$ words before it (n-gram model)

$$\begin{aligned} P(w_m | w_1 w_2 w_3 \dots w_{m-1}) &= P(w_m | w_1, w_2, w_3, \dots, w_{m-1}) \\ &= P(w_m | w_{m-n} w_{m-n+1} w_{m-n+2} \dots w_{m-1}) \end{aligned}$$

N-gram Language Models

- Unigram model: $P(w_1 w_2 \dots w_n) \sim \prod_i P(w_i)$
- Bigram model: $P(w_1 w_2 \dots w_n) \sim \prod_i P(w_i | w_{i-1})$
- Trigram model: $P(w_1 w_2 \dots w_n) \sim \prod_i P(w_i | w_{i-1} w_{i-2})$

Estimating bigram probabilities

- The Maximum Likelihood Estimate

$$P(w_i \mid w_{i-1}) = \frac{\textit{count}(w_{i-1}, w_i)}{\textit{count}(w_{i-1})}$$

$$P(w_i \mid w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

Example 1

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

$$P(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

$$P(\text{I} | \text{<s>}) = \frac{2}{3} = .67$$

$$P(\text{Sam} | \text{<s>}) = \frac{1}{3} = .33$$

$$P(\text{am} | \text{I}) = \frac{2}{3} = .67$$

$$P(\text{</s>} | \text{Sam}) = \frac{1}{2} = 0.5$$

$$P(\text{Sam} | \text{am}) = \frac{1}{2} = .5$$

$$P(\text{do} | \text{I}) = \frac{1}{3} = .33$$

Exercise

Given the following corpus:

<s> I am Sam </s>

<s> Sam I am </s>

<s> Sam I like </s>

<s> Sam I do like </s>

<s> do I like Sam </s>

Predict the next word in the following phrase:

<s> Sam ... </s>

<s> Sam I do ... </s>

<s> Sam I am Sam ... </s>

<s> do I like ... </s>

Exercise

Given the following corpus:

<s> I am Sam </s>

<s> Sam I am </s>

<s> Sam I like </s>

<s> Sam I do like </s>

<s> do I like Sam </s>

a. Predict the next word in the following phrase:

<s> Sam ... </s>

<s> Sam I do ... </s>

<s> Sam I am Sam ... </s>

<s> do I like ... </s>

b. Sử dụng mô hình bigram, đánh giá câu nào trong các câu sau đây tốt hơn (xác suất cao hơn)

<s> Sam I do I like </s>

<s> Sam I am </s>

<s> I do like Sam I am </s>

Raw bigram counts

- Out of 9222 sentences

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

$c(\text{"want to"}) = 608$

Raw bigram probabilities

- Normalize by unigrams:

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

- Result:

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

- $P(\text{english} | \text{want}) = .0011$
- $P(\text{chinese} | \text{want}) = .0065$
- $P(\text{to} | \text{want}) = .66$
- $P(\text{eat} | \text{to}) = .28$
- $P(\text{food} | \text{to}) = 0$
- $P(\text{want} | \text{spend}) = 0$
- $P(i | \langle s \rangle) = .25$

Bigram estimates of sentence probabilities

$$\begin{aligned} P(<s> \text{ I want english food } </s>) = \\ & P(\text{I} | <s>) \\ & \times P(\text{want} | \text{I}) \\ & \times P(\text{english} | \text{want}) \\ & \times P(\text{food} | \text{english}) \\ & \times P(</s> | \text{food}) \\ & = .000031 \end{aligned}$$

Practical Issues

- We do everything in log space
 - Avoid underflow
 - (also adding is faster than multiplying)

$$\log(p_1 \square p_2 \square p_3 \square p_4) = \log p_1 + \log p_2 + \log p_3 + \log p_4$$

Language Modeling Toolkits

- Google Book N-grams
 - <https://books.google.com/ngrams/>
- KenLM
 - <https://kheafield.com/code/kenlm/>

Google Book N-grams

Google Books Ngram Viewer

LLM,GPT,VLM

×

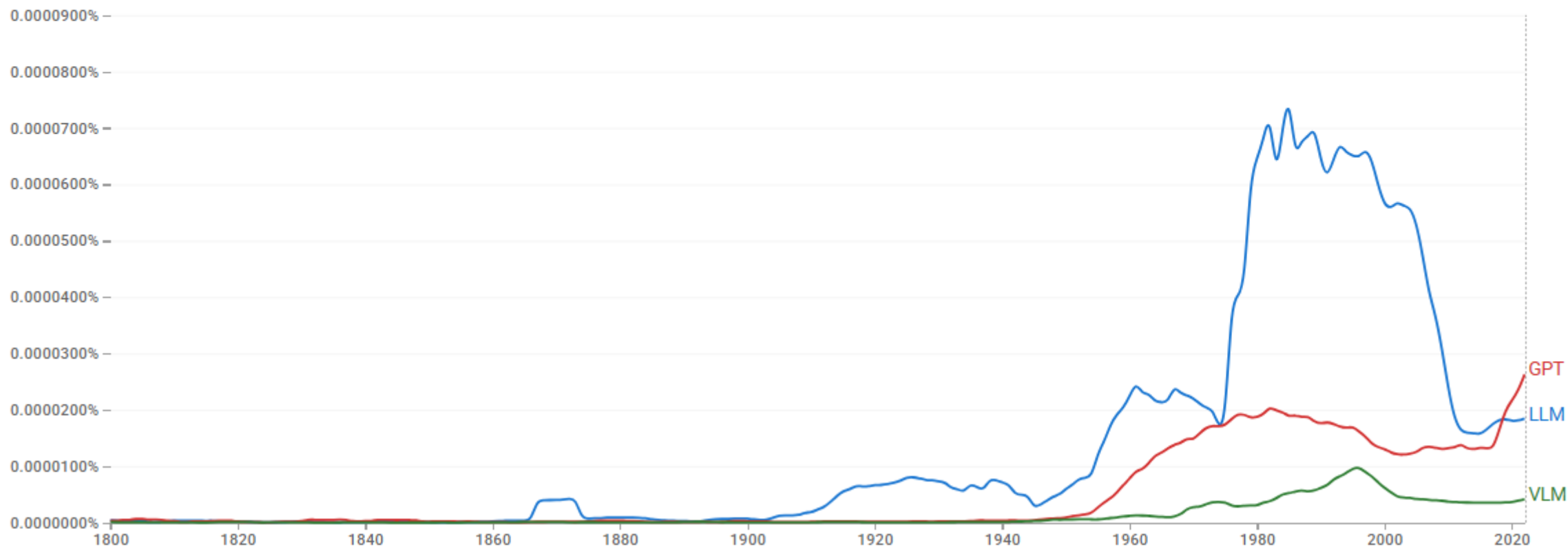
?

1800 - 2022

English

Case-Insensitive

Smoothing





[Scalable Modified Kneser-Ney Language Model Estimation](#)

Kenneth Heafield, Ivan Pouzyrevsky, Jonathan H. Clark, and Philipp Koehn. [ACL](#), Sofia, Bulgaria, 4—9 August, 2013.

[\[Paper\]](#) [\[Slides\]](#) [\[BibTeX\]](#)

Querying is [fast and low-memory](#), as shown in the paper

[KenLM: Faster and Smaller Language Model Queries](#)

Kenneth Heafield. [WMT at EMNLP](#), Edinburgh, Scotland, United Kingdom, 30—31 July, 2011.

[\[Paper\]](#) [\[Slides\]](#) [\[BibTeX\]](#)

Usage

Moses, cdec, Joshua, Jane, and Phrasal already distribute KenLM and build it along with the decoder. See their documentation on where to find the programs. Estimation and filtering require [Boost](#) at least 1.36.0 and zlib. I also recommend tcmalloc from [gperftools](#), bzlib (to read .bz2 files), and xz-utils (to read .xz files). If your distribution has "devel" packages, install those. For more help, see [dependencies](#). Compilation follows the standard `cmake` process, for which we recommend a separate build directory:

```
wget -O - https://kheafeld.com/code/kenlm.tar.gz | tar xz
mkdir kenlm/build
cd kenlm/build
cmake ..
make -j2
```

If the dependencies are too difficult and you only need querying, use `./compile_query_only.sh` that depends only on `g++` and `bash`. Programs will be located in `bin/`.

Estimating

Google N-Gram Release

- serve as the incoming 92
- serve as the incubator 99
- serve as the independent 794
- serve as the index 223
- serve as the indication 72
- serve as the indicator 120
- serve as the indicators 45
- serve as the indispensable 111
- serve as the indispensable 40
- serve as the individual 234

<http://googleresearch.blogspot.com/2006/08/all-our-n-gram-are-belong-to-you.html>

Evaluation: How good is our model?

- Does our language model prefer good sentences to bad ones?
 - Assign higher probability to “real” or “frequently observed” sentences
 - Than “ungrammatical” or “rarely observed” sentences?
- We train parameters of our model on a **training set**.
- We test the model’s performance on data we haven’t seen.
 - A **test set** is an unseen dataset that is different from our training set, totally unused.
 - An **evaluation metric** tells us how well our model does on the test set.

Extrinsic evaluation of N-gram models

- Best evaluation for comparing models A and B
 - Put each model in a task
 - spelling corrector, speech recognizer, MT system
 - Run the task, get an accuracy for A and for B
 - How many misspelled words corrected properly
 - How many words translated correctly
 - Compare accuracy for A and B

Intrinsic evaluation

- **Intrinsic** evaluation using **perplexity** (độ phức tạp)
 - Bad approximation
 - unless the test data looks **just** like the training data
 - So **generally only useful in pilot experiments**

Intuition of Perplexity

- **The Shannon Game:**

- How well can we predict the next word?

I always order pizza with cheese and _____

The 33rd President of the US was _____

I saw a _____

- Unigrams are terrible at this game. (Why?)
- A better model of a text
 - is one which assigns a higher probability to the word that actually occurs

mushrooms 0.1

pepperoni 0.1

anchovies 0.01

....

fried rice 0.0001

....

and 1e-100

Perplexity

The best language model is one that best predicts an unseen test set

- Gives the highest $P(\text{sentence})$

Perplexity is the inverse probability of the test set, normalized by the number of words:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_1 \dots w_{i-1})}}$$

Chain rule:

For bigrams:

$$PP(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

$$\begin{aligned} PP(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \end{aligned}$$

Minimizing perplexity is the same as maximizing probability

Perplexity as branching factor

- Let's suppose a sentence consisting of random digits (0÷9)
- What is the perplexity of this sentence according to a model that assign $P=1/10$ to each digit?

$$\begin{aligned} \text{PP}(W) &= P(w_1 w_2 \dots w_N)^{-\frac{1}{N}} \\ &= \left(\frac{1}{10}\right)^{-\frac{1}{N}} \\ &= \frac{1}{10}^{-1} \\ &= 10 \end{aligned}$$

$$\text{PP}(W) = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i|w_{i-1})}}$$

Lower perplexity = better model

- Training 38 million words, test 1.5 million words, WSJ

N-gram Order	Unigram	Bigram	Trigram
Perplexity	962	170	109

The perils of overfitting

- N-grams only work well for word prediction if the test corpus looks like the training corpus
 - We need to train robust models that generalize!
 - One kind of generalization: Zeros!
 - Things that don't ever occur in the training set but occur in the test set

- Training set:
 - ... denied the allegations
 - ... denied the reports
 - ... denied the claims
 - ... denied the request
- Test set
 - ... denied the offer
 - ... denied the loan

$$P(\text{"offer"} \mid \text{denied the}) = 0$$

→ Probability of a sentence or a sequence of words = 0

➤ Using smoothing

The intuition of smoothing (from Dan Klein)

- When we have sparse statistics:

$P(w \mid \text{denied the})$

3 allegations

2 reports

1 claims

1 request

7 total

- Steal probability mass to generalize better

$P(w \mid \text{denied the})$

2.5 allegations

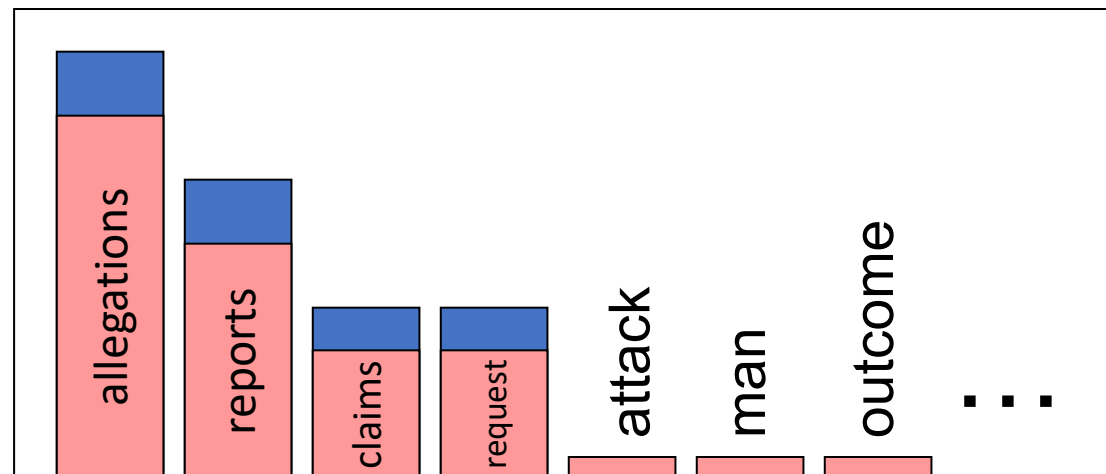
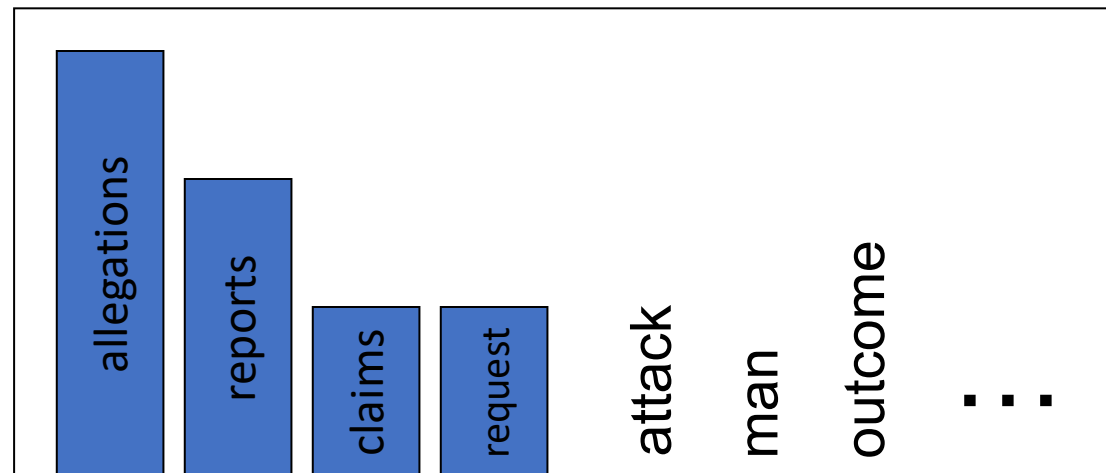
1.5 reports

0.5 claims

0.5 request

2 other

7 total



Add-one estimation

- Also called Laplace smoothing
- Pretend we saw each word one more time than we did
- Just add one to all the counts!

- MLE estimate:
$$P_{MLE}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i)}{c(w_{i-1})}$$

- Add-1 estimate:
$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}, w_i) + 1}{c(w_{i-1}) + V}$$